

Algorytmy Grafowe – Kompendium na Kolokwium

1 Reprezentacja grafu

- **Lista krawędzi:** Tablica krotek, np. $[(u, v, w), \dots]$. Często stosowana jako wejście w zadaniach lub w algorytmie Kruskala.
- **Reprezentacja macierzowa:** Tablica $n \times n$. Jeśli jest krawędź z v do w , to $\text{tab}[v][w] = \text{waga}$ (lub True jak krawędzie nie mają wag). Brak krawędzi to np. 0.
- **Listy sąsiedztwa (najczęstsza):** W $\text{tab}[i]$ mamy listę wierzchołków (oraz wag), do których prowadzi krawędź z i . Idealna dla BFS, DFS, Dijkstry. Złożoność większości algorytmów zależy od tego, że możemy szybko przejrzeć tylko istniejące krawędzie z danego wierzchołka.

2 Przeszukiwanie grafu

2.1 BFS (Przeszukiwanie wszerz)

Wykorzystuje kolejkę (np. `collections.deque`). Idealny do znajdowania **najkrótszej ścieżki w grafach nieważonych** (gdzie każda krawędź ma wagę 1). Złożoność: $O(V + E)$.

2.2 DFS (Przeszukiwanie w głąb)

Wykorzystuje rekurencję (lub stos). Podstawa do wielu zaawansowanych algorytmów (sortowanie topologiczne, spójne składowe, mosty). Złożoność: $O(V + E)$.

3 Zaawansowane zastosowania DFS

- **Sortowanie topologiczne (DAG):** Uruchamiamy DFS. W momencie "przetworzenia" wierzchołka (gdy odwiedzimy wszystkich jego sąsiadów i kończymy w nim wywołanie rekurencyjne), dopisujemy go **na początek** tworzonej listy (kolejności). *Zastosowanie na kolokwiach:* Zadania z zależnościami, np. harmonogramowanie projektów, gdzie projekt A musi być wykonany przed projektem B.
- **Cykl Eulera:** Przejście przez wszystkie krawędzie grafu dokładnie raz. Graf nieskierowany ma cykl Eulera wtw. gdy jest spójny i każdy wierzchołek ma stopień **parzysty**. *Algorytm:* DFS, w którym na bieżąco usuwamy odwiedzane krawędzie. W momencie, gdy z wierzchołka nie wychodzą już żadne krawędzie, dopisujemy go na początek cyklu. Uwaga: dla list sąsiedztwa warto mieć je posortowane, jeśli wymagana jest konkretna kolejność leksykograficzna.

- **Silnie Spójne Składowe (SCC - Kosaraju):** 1. Uruchom DFS i zapisz czasy przetworzenia wierzchołków. 2. Odwróć kierunek wszystkich krawędzi w grafie. 3. Wykonaj DFS ponownie, analizując wierzchołki w kolejności malejących czasów przetworzenia z kroku 1.
- **Mosty (krawędzie, których usunięcie rozspójnia graf):** Wykonujemy DFS, zapisując czasy odwiedzenia $d(v)$.
Dla każdego wierzchołka liczymy:
 $low(v) = \min(d(v), \min d(u) \text{ dla krawędzi wstecznych}, \min low(w) \text{ dla dzieci w drzewie DFS})$.
Krawędź $\{parent(v), v\}$ jest mostem, gdy $low(v) = d(v)$.

4 Najkrótsze ścieżki w grafach ważonych

4.1 Algorytm Dijkstry

Służy do szukania najkrótszych ścieżek z jednego źródła do pozostałych wierzchołków.

Warunek: Brak krawędzi o wagach ujemnych.

Złożoność: $O(E \log V)$.

Istotny niuans implementacyjny (optymalizacja): Zgodnie z wytycznymi z wykładów, **nie należy** wrzucać na początku wszystkich wierzchołków z wagą ∞ do kolejki priorytetowej. Zamiast tego:

1. Inicjalizujemy tablicę odległości d wartościami ∞ , a odległość dla startu $s.d = 0$.
2. Do kolejki wrzucamy tylko wierzchołek startowy.
3. Gdy wykonujemy relaksację krawędzi (u, v) i znajdujemy krótszą ścieżkę ($u.d + w(u, v) < v.d$), uaktualniamy $v.d$ i **dodajemy wierzchołek v ponownie do kolejki** z nową, mniejszą wagą. Wierzchołki wyciągnięte z kolejki z wagą większą niż ich aktualne $v.d$ po prostu ignorujemy.

4.2 Algorytm Bellmana-Forda

Dopuszcza wagi ujemne (tylko w grafach skierowanych, inaczej krawędź ujemna tworzy ujemny cykl).

Złożoność: $O(V \cdot E)$.

Algorytm: $V - 1$ razy przechodzimy przez wszystkie krawędzie w grafie i wykonujemy dla nich relaksację. Na koniec wykonujemy dodatkowe, V -te przejście – jeśli jakkolwiek relaksacja nadal zmniejsza dystans, oznacza to istnienie **cyklu o ujemnej wadze**.

5 Minimalne Drzewo Rozpinające (MST)

Szukamy podzbioru krawędzi grafu nieskierowanego, który łączy wszystkie wierzchołki (tworzy drzewo) i ma minimalną sumę wag.

5.1 Algorytm Kruskala

1. Sortujemy wszystkie krawędzie grafu rosnąco po wagach. Złożoność sortowania to $O(E \log V)$.
2. Przechodzimy przez posortowane krawędzie. Jeśli krawędź łączy dwa różne zbiory wierzchołków (nie tworzy cyklu), dodajemy ją do MST.

Niuans implementacyjny (Find-Union / Lasy Zbiorów Rozłącznych): Aby szybko sprawdzać cykle, używamy struktury Find-Union opartej na tablicach **parent** i **rank**. Kluczowe optymalizacje, które obniżają złożoność operacji do $O(\log^* V)$ (logarytm iterowany, w praktyce $O(1)$):

- **Kompresja ścieżki w `find_set(x)`:** Podczas szukania reprezentanta zbioru, przepi-
namy `x.parent` (oraz wszystkich przodków po drodze) bezpośrednio na znalezionego
reprezentanta głównego.
- **Łączenie według rangi (Union by rank):** W funkcji `union(x, y)`, mniejsze drzewo
(o mniejszej randze) dołączamy pod korzeń wyższego drzewa. Jeśli rangi są równe, wy-
bieramy dowolny korzeń i zwiększamy jego rangę o 1. Zapewnia to płytkość drzew.

5.2 Algorytm Prima

Buduje MST podobnie jak Dijkstra szuka ścieżek. Zaczynamy od dowolnego wierzchołka. Trzy-
mamy w kolejce priorytetowej wierzchołki, a kluczem jest waga najtańszej krawędzi łączącej
dany wierzchołek z aktualnie zbudowanym fragmentem drzewa. W każdym kroku wyjmujemy
najtańszy wierzchołek i aktualizujemy koszty dotarcia do jego sąsiadów.

6 Motywy z zadań z poprzednich kolokwiów (Wskazówki)

Na podstawie historycznych zadań (m.in. *Warrior*, *SpaceTravel*, *Let's Roll*, *Beautree*) powta-
rzają się konkretne schematy:

1. **Grafy z rozszerzonym stanem (Grafy warstwowe / Dijkstra na stanach):** Czę-
sty motyw to szukanie najkrótszej trasy, ale z pewnym ograniczeniem (np. "kierowca
musi zmienić się co 3 postoje", "wojownik musi odpocząć w schronisku po 16h", "ma-
giczne zagięcia czasoprzestrzeni"). *Rozwiązanie:* Wierzchołkiem w algorytmie (np. w ko-
lejce Dijkstry) nie jest tylko `numer_miasta`, ale krotka ze **stanem**, np. (`odleglosc`, `u`,
`aktualne_zmeczenie`) lub (`odleglosc`, `u`, `czy_wykorzystano_magie`). Tablica odle-
głości staje się wtedy tablicą 2D lub słownikiem: `dist[u][stan]`.
2. **Modyfikacje MST (Kruskala):** Zadania typu: znajdź drzewo rozpinające, w którym
różnica między najdroższą a najtańszą krawędzią jest specyficzna (tzw. "piękne drzewo").
Rozwiązanie: Ponieważ krawędzi jest zazwyczaj mało (złożoność $O(E^2)$ jest często akcep-
towalna), można posortować krawędzie rosnąco, a następnie próbować budować MST z
algorytmu Kruskala, wymuszając by najmniejszą krawędzią drzewa była i -ta krawędź z
posortowanej listy (pętla po E , w środku uruchamiamy Kruskala).
3. **Zależności i harmonogramy:** Zadania mówiące, że "projekt A musi być wykonany
przed B", a możemy wykonywać dowolnie wiele naraz. *Rozwiązanie:* Jest to klasyczny
graf skierowany i **Sortowanie Topologiczne**. Jeśli wierzchołek zależy od innych, jego
najwcześniejszy czas ukończenia to $\max(\text{czasy ukończenia poprzedników}) + 1$.